

Evaluating Pre-trained Large Language Models on Zero Shot Prompts for Parallelization of Source Code

Devansh Yadav, Shouvick Mondal*

*Software Engineering and Testing (SET) Group
Indian Institute of Technology Gandhinagar, Gujarat, India*

Abstract

Large Language Models (LLMs) have recently gained prominence as intelligent software tools and are used in most phases of the software development life cycle (SDLC). While source code generation has been widely studied, the generation of performant source code has received very little research attention. Although automatic code parallelization tools exist in the non-LLM setting, their analogue in the context of transformation using pre-trained models requires investigation. In this work, we perform a comparative study of 23 pre-trained LLMs considering the state-of-the-art Intel C Compiler (icc) as the baseline auto-parallelization tool. Our motivation is to investigate how well LLMs play the role of an auto-parallelizer, given a sequential C source code. Our experiments show that, in some instances, LLM-based zero-shot source code parallelization outperforms icc in non-functional aspects, such as speedup. Using 30 kernels from the popular PolyBench C benchmarks, we evaluated 23 pre-trained LLMs, generating 667 parallelized code versions. We identified notable performance gains after filtering out those with compilation errors or data race issues through comprehensive memory and threading error analysis. Across all tested LLMs, 26.66% of cases achieved a speedup surpassing that of the state-of-the-art (icc). The best-performing LLM-generated code achieved a parallelization speedup of $7.5\times$ compared to the icc-parallelized version (speedup of $1.08\times$). Our study shows that pre-trained LLMs tasked with zero shot sequential to parallel code translation, i.e., even without fine-tuning or chain-of-thought prompting (or feedback prompts), perform competitively with non-LLM based compiler technology that has been developed over the past decades.

Keywords: Large Language Models, Zero-Shot, Code Parallelization, Compiler Optimization

1. INTRODUCTION

The growing complexity of computational problems has driven a greater reliance on parallel computing to enhance performance. By distributing computational workloads, parallelization allows programs to fully leverage modern multi-core architectures, significantly reducing execution time. The PolyBench benchmark suite [1], which includes compute-intensive applications from domains like linear algebra, data mining, and stencils, provides a standardized platform for evaluating parallelization strategies. Efficiently parallelizing these benchmarks is crucial for high-performance computing (HPC) environments, as it not only shortens execution time but also preserves computational accuracy.

*Corresponding author

Email addresses: devansh.yadav@iitgn.ac.in (Devansh Yadav), shouvick.mondal@iitgn.ac.in (Shouvick Mondal)

However, parallelization is a complex process. Challenges such as managing data dependencies, ensuring correct synchronization, and avoiding concurrency issues like data races and memory access errors make it complex. Traditional tools like the Intel C Compiler (ICC) [2] have been widely used for automatic parallelization, leveraging advanced compiler optimizations to distribute workloads across multiple threads relatively quickly. Recently, the emergence of large language models (LLMs) has introduced a new avenue for automatic code generation [3, 4], including parallel code generation [5] from multi-shot prompts involving Natural Language (NL) instructions and exemplars. However, to the best of our knowledge and understanding, LLMs’ ability to correctly handle the intricacies of parallel programming—such as managing synchronization, memory, and data races—has not yet been fully explored [4, 6] in the *zero shot* setting. While promising, LLM-generated parallel code can still be prone to errors, particularly in complex parallel computing scenarios. From the perspective of LLM-based sequential to parallel code translation, Mišić and Dodović [7] recently conducted a human-in-the-loop (multi-turn) prompt feedback driven generation of parallel code for a fixed parallelization factor. However, the generated/translated code when executed with different parallelization factors (i.e., more/less number of threads on the same hardware) may yield different speedups. Whether LLMs are capable of translating sequential to performant parallel code that scales with the available parallelization factor is worthy of investigation. We believe this is an important research gap, and unaware of any existing work that address the same.

In this work, we show that *pre-trained* LLMs tasked with *zero shot sequential to parallel code translation*, i.e., even without fine-tuning or chain-of-thought prompting [8] (or feedback prompts [9]), perform competitively with non-LLM based compiler technology that has been developed over the past decades.

1.1. Definitions and Preliminaries

We now state some definitions and preliminary concepts used in our work.

Parallelization: Parallelization is the process of dividing computational tasks into smaller sub-tasks that can be executed concurrently on multiple processing units [10]. This strategy aims to optimize performance, primarily by reducing execution time in multi-core architectures.

Serial to Parallel Code Transformation: The transformation from serial to parallel code involves rewriting code that runs sequentially into a version that can be executed in parallel. This typically includes handling data dependencies, synchronization mechanisms, and ensuring that shared resources are properly managed across threads [11].

Large Language Models (LLMs): LLMs, such as GPT-3.5 and GPT-4, are pre-trained artificial intelligence models that can perform tasks including code generation. These models have the potential to transform serial code into parallel code automatically but often struggle with complex parallelization nuances, such as reduction variables and loop dependencies.

Reduction Loops: Reduction loops are a type of loop where an operation (e.g., summation) is applied repeatedly to accumulate results across iterations [12]. In parallelization, these loops require special handling to avoid incorrect results due to concurrent modifications of shared variables.

Inter-Iteration Dependencies: These occur when the outcome of one iteration of a loop depends on the results of previous iterations. Such dependencies can prevent a loop from being parallelized because breaking the order of execution might result in incorrect computations [13].

Data Races: A data race happens when two or more threads access shared data simultaneously, with at least one of the threads modifying it. This can lead to unpredictable behaviour or incorrect program results. Detecting and resolving data races is crucial in ensuring the correctness of parallel programs [14].

To illustrate, consider the example of the *adi* kernel in the PolyBench suite, which involves nested loops iterating over row and column sweeps. The *adi* (*Alternating Direction Implicit*) kernel in PolyBench is used to solve partial differential equations using the Alternating Direction Implicit method. This method

is particularly useful for solving time-dependent problems, such as heat conduction or diffusion equations, where the solution evolves over time. Forward substitution is used to solve a lower triangular system of equations. Backward substitution is used to solve an upper triangular system of equations. When tasked with parallelizing this kernel, ICC reported that the loop could not be parallelized due to output dependence, i.e., write-after-write (WAW). Interestingly, LLMs produced different implementations with distinct strengths and weaknesses. Although all models successfully inserted OpenMP directives to parallelize the loops, data races and memory errors were prevalent in some instances. Upon analyzing these implementations using Intel Inspector, we identified problems such as threading-related data races and uninitialized memory access. GPT-4o generated correct parallel code, and Claude-3-Haiku not only produced correct code but also achieved a 6x speedup. However, GPT-3.5 Turbo generated incorrect parallelized code, leading to six threading-related issues and three memory-related problems.

Listing 1: Original C Code (sequential) for kernel adi from PolyBench/C 4.2 [15]

```

1 ...
2 /* Main computational kernel. */
3 static void kernel_adi(...)
4 {
5 ...
6     for (t=1; t<=_PB_TSTEPS; t++) {
7         ...
8     }
9 ...
10 }
11
12 int main(int argc, char** argv)
13 {
14     ...
15     kernel_adi (...);
16     ...
17     return 0;
18 }

```

Listing 2: Correct Parallelization by GPT-4o-2024-05-13 for kernel adi from PolyBench/C 4.2

```

1 ...
2 for (t=1; t<=_PB_TSTEPS; t++) {
3     //Column Sweep
4     #pragma omp parallel for private(i, j)
5     for (i=1; i<_PB_N-1; i++) {
6         ...
7     }
8     //Row Sweep
9     #pragma omp parallel for private(i, j)
10    for (i=1; i<_PB_N-1; i++) {
11        ...
12    }
13 ...

```

Listing 3: Correct Parallelization by claude-3-haiku@20240307 for kernel adi from PolyBench/C 4.2

```

1 ...
2 #pragma omp parallel for private(i, j)

```

```

3 for (t = 1; t <= _PB_TSTEPS; t++) {
4     // Column Sweep
5     #pragma omp for
6     for (i = 1; i < _PB_N - 1; i++) {
7         ...
8     }
9     // Row Sweep
10    #pragma omp for
11    for (i = 1; i < _PB_N - 1; i++) {
12        ...
13    }
14    ...

```

Listing 4: Incorrect Parallelization by GPT-3.5 Turbo for kernel adi from PolyBench/C 4.2

```

1 ...
2 /* Main computational kernel. */
3 static void kernel_adi(...)
4 {
5     ...
6     #pragma omp parallel for private(t, i, j)
7     for (t=1; t<=_PB_TSTEPS; t++) {
8         //Column Sweep
9         #pragma omp parallel for private(i, j)
10        for (i=1; i<_PB_N-1; i++){
11            v[0][i] = ...;
12            p[i][0] = ...;
13            q[i][0] = v[0][i];
14            for (j=1; j<_PB_N-1; j++) {
15                p[i][j] = -c / (a*p[i][j-1]+b);
16                q[i][j] = (-d*u[j][i-1]+(...*u[j][i] - f*u[j][i+1]-a*q[i][j-1]))/(a*p[i][j-1]+b);
17            }
18            v[_PB_N-1][i] = ...;
19            for (j=_PB_N-2; j>=1; j--) {
20                v[j][i] = p[i][j] * v[j+1][i] + q[i][j];
21            }
22        }
23        //Row Sweep
24        #pragma omp parallel for private(i, j)
25        for (i=1; i<_PB_N-1; i++) {
26            u[i][0] = ...;
27            p[i][0] = ...;
28            q[i][0] = u[i][0];
29            for (j=1; j<_PB_N-1; j++) {
30                p[i][j] = -f / (d*p[i][j-1]+e);
31                q[i][j] = (-a*v[i-1][j]+(...*a)*v[i][j] - c*v[i+1][j]-d*q[i][j-1]))/(d*p[i][j-1]+e);
32            }
33            u[i][_PB_N-1] = ...;
34            for (j=_PB_N-2; j>=1; j--) {
35                u[i][j] = p[i][j] * u[i][j+1] + q[i][j];
36            }
37        }
38    }
39 }
40 ...

```

```

41 }
42
43 int main(int argc, char** argv)
44 {
45     ...
46     kernel_adi (...);
47     ...
48     return 0;
49 }

```

The analysis of the Listing 4 reveals multiple data race conditions. These data races occur primarily in the `kernel_adi` function, which is responsible for performing the ADI (*Alternating Direction Implicit*) method. The data races are identified during both the column and row sweeps of the algorithm. Below is a summary of the key findings:

Column Sweep Data Races:

- In loop starting at *Line 10*: Multiple threads write to `p[i][j]` and `q[i][j]` concurrently.
- In loop starting at *Line 14*: Concurrent writes and reads to/from `u[i][j]` during the initialization phase.
- *Line 31*: Concurrent writes and reads to/from `v[j][i]` during the backward substitution phase.

Row Sweep Data Races:

- In loop starting at *Line 20*: Concurrent writes and reads to/from `v[i][j]` during the initialization phase.
- In loop starting at *Line 26*: Concurrent writes and reads to/from `u[i][j]` during the backward substitution phase.
- *Line 30*: Concurrent writes to `p[i][j]` and `q[i][j]` during the forward substitution phase.
- In loop starting at *Line 35*: Concurrent writes and reads to/from `u[i][j]` during the backward substitution phase.

These examples demonstrate that, despite the advanced capabilities of LLMs, they can still encounter difficulties in generating accurate parallel code. This underscores the critical need for thorough validation when using automated tools for code parallelization.

Despite advancements in compiler technology, generating efficient parallel code from serial programs presents significant challenges. Achieving optimal speedup without introducing errors such as data races or memory leaks remains a persistent issue. This study specifically examines the challenges of using the Intel C Compiler (ICC) for parallelizing PolyBench benchmarks and explores alternative approaches using large language models (LLMs) to automate parallel code generation. Evaluating the effectiveness of these methods in terms of speedup, memory, and threading accuracy forms the crux of this research. In particular, we seek to answer the three following research questions (RQs):

RQ1 (Parallelization and Concurrency Bugs): *What are the common pitfalls and challenges faced by LLMs in generating correct parallel code, particularly regarding data races and synchronization? What are the memory and threading error rates in LLM-generated parallel code, and how do they compare to state-of-the-art? This question focuses on understanding the limitations of LLMs in parallel programming and the types of errors they frequently produce. It also investigates the reliability of LLM-generated code*

by analyzing the frequency of memory issues and threading errors. This also implies how frequently the generated parallel codes are functionally correct.

RQ2 (Speedup): *How does the performance of LLM-generated parallel code compare to that of the state-of-the-art in terms of speedup?* This question aims to evaluate the comparative speedup achieved by LLM-generated parallel code versus ICC-parallelized code. We identify the best-performing LLMs and the features that enable them to generate efficient parallel code.

This paper presents the following key contributions:

- A thorough evaluation of the Intel C Compiler’s (icc) auto-parallelization capabilities, analyzing the speedup achieved across the PolyBench C benchmarks.
- To the best of our knowledge, a novel assessment of how pre-trained LLMs can be utilized to automate the parallelization of sequential code (provided as a zero shot prompt), and comparison of their performance against state-of-the-art compiler based auto-parallelization.
- A detailed examination of data race and memory issues encountered in LLM-generated parallel code offering valuable insights into the reliability and robustness of AI-driven parallelization.
- A comparative analysis between LLM-generated and compiler-optimized parallel code, focusing on factors such as performance, accuracy.
- A dataset¹ [16] containing complete code, data, and scripts used to conduct our experiments.

The rest of this paper is organized as follows. Section 2 mentions relevant related work. Section 3 explains methods and techniques. Section 4 outlines tools and software environment. Section 5 elaborates framed research questions. Section 6 discuss the actionable implications and potential downstream use cases. Section 7 discusses threads to validity and limitation of our approach. Section 8 states directions for future work. Finally, Section 9 concludes the paper.

2. RELATED WORK

In this section, we mention the most relevant related work in the existing literature and perform a qualitative comparison.

2.1. PolyBench and Its Applications

PolyBench, a suite of benchmarks primarily designed to evaluate the performance of compilers and optimization techniques, has been extensively used in research to analyze how compilers handle different workloads [1]. The suite consists of compute-intensive kernels from various domains, such as linear algebra, stencils, and datamining, and it offers various dataset sizes to explore scalability [17]. PolyBench has been leveraged to investigate the performance of different compiler optimizations across architectures, making it an essential tool for studying the impacts of parallelization strategies on execution time [18]. Recent research highlights the use of PolyBench to compare different parallelization approaches and evaluate system-level performance on multi-core platforms [19].

Our research highlights PolyBench as a valuable tool for evaluating code parallelization driven by large language models (LLMs), an area that has not been extensively explored. It provides insights into how

¹This will be made public upon approval from OpenAI.

LLMs address parallelization issues, particularly focusing on data races and synchronization challenges. This adds depth to research based on PolyBench. Consequently, the application of PolyBench extends beyond traditional compiler assessments, establishing it as a useful framework for testing emerging AI-driven methodologies.

2.2. Automatic Parallelization

Automatic parallelization tools and techniques have seen significant development, aimed at minimizing the manual effort required to parallelize serial code [20]. Tools like OpenMP and auto-parallelization features in modern compilers, such as ICC, enable the automatic transformation of loops and other parallelizable segments into multi-threaded code [21]. Studies focusing on PolyBench have demonstrated the effectiveness of automatic parallelization in improving performance, particularly in multi-core environments [22]. However, it's important to note that challenges such as load balancing, memory access patterns, and data dependencies can hinder the effectiveness of automatic tools [23]. Research has shown that while automatic parallelization can reduce execution times, the resultant speedup is often architecture-dependent [24]. In the space of autonomous parallelization of source code, a recent work by Mišić and Dodović [7] evaluated 9 kernels with a constant factor runtime parallelization of 8 threads. However, they considered only two specific LLMs: ChatGPT 3.5, and GitHub Copilot. In contrast, our study considered 30 kernels from the popular Polybench C-4.2 benchmarks. Our evaluation considered 23 LLMs (both open-source and closed-source including different versions of ChatGPT, Gemini, Claude) and a *varying* runtime parallelization factor (#threads): {2, 4, 8, 12, 16, 20, 24} (*c.f.* Section 4.1).

Our research differentiates itself by exploring how well LLMs autonomously rewrite and parallelize code, particularly for cases where dependency complexities would otherwise require significant manual intervention. Evaluating the stability and error rates of LLM-generated code in parallel settings, thus contributing to a new understanding of LLMs' capabilities as reliable tools for automatic parallelization, especially in contexts where traditional tools encounter limitations.

2.3. Code Generation Challenges

Maintaining both accuracy and efficiency during code parallelization is a fundamental challenge [25]. The primary obstacles include managing data dependencies, handling synchronization, and avoiding race conditions [26]. For example, LLMs (Large Language Models) are being explored as potential tools for automating code generation, including parallelization [27]. While promising, LLM-generated code faces challenges in ensuring correctness, particularly with respect to race conditions and memory handling [28]. Studies exploring these models highlight the crucial need for robust verification techniques, such as data race detection and memory analysis, to ensure that parallel code runs correctly on multi-threaded architectures [29].

Our research contributes by analyzing how well LLM-generated code meets requirements for accuracy, efficiency, and correctness in parallel environments, focusing on common challenges like data races and memory handling. Utilizing Intel Inspector and other verification tools to rigorously check LLM-generated code for concurrency issues, assessing whether LLMs can generate dependable code for multi-threaded applications.

2.4. Compiler Optimizations

Compiler optimizations play a critical role in generating efficient parallel code. Techniques such as loop unrolling, vectorization, and instruction-level parallelism (ILP) are widely used to optimize execution [13]. Modern compilers like ICC provide advanced optimization reports detailing how parallelization

is achieved [2]. These optimizations, however, are not always perfect—certain benchmarks may not see significant speedup due to factors like poor cache utilization or high synchronization costs [30]. The literature on compiler optimizations, especially within the PolyBench suite, shows that while ICC’s parallelization techniques are powerful, they may introduce bottlenecks when handling irregular memory access patterns or fine-grained parallelism [1].

Our research compares the performance of LLM-generated parallel code with compiler-optimized code to see if LLMs can better manage cases where compiler optimizations are less effective, such as handling complex or irregular memory access. Investigating whether LLMs’ adaptive capabilities can mitigate issues like bottlenecks caused by conventional optimization techniques, offering an alternative for managing challenging code structures.

3. METHODOLOGY

This section provides a detailed explanation of the methods and techniques used in the parallelization of the PolyBench benchmark suite.

3.1. Approach Overview

This section outlines the strategy used to parallelize the PolyBench benchmark suite. We focus on systematically transforming serial code into parallelized versions using both traditional compiler techniques and modern approaches, such as utilizing Large Language Models (LLMs) [27]. This dual methodology allows for a comprehensive evaluation of the effectiveness of compiler optimizations versus LLM-assisted code generation.

3.2. Serial to Parallel Code Transformation

The serial-to-parallel transformation begins with compiling PolyBench’s serial code using the Intel C Compiler (ICC). The code is optimized for performance with flags such as `-O3` and parallelized using `-parallel` [13]. For further analysis, the benchmarks are run on multiple threads (2, 4, 8, 12, 16, 20, 24), capturing the execution times for comparison with parallelized versions. Additionally, the script averages the middle three execution times of five benchmark runs to ensure accuracy in performance measurement.

To introduce an alternative approach, LLM-generated parallel code is also created from the original serial versions. This involves using LLMs to automatically rewrite the serial code to its parallelized counterpart, which is later evaluated for efficiency and correctness.

3.3. Optimization Techniques

The primary optimization techniques applied during parallelization include ICC’s automatic parallelization (`-parallel`) optimization with OpenMP directives [21]. Compiler reports are generated using `-qopt-report-phase=par` and `-qopt-report:5` to evaluate the effectiveness of parallelization and identify potential bottlenecks [2].

For LLM-generated parallel code, we rely on further optimization by applying the same ICC flags and running Intel Inspector to identify and eliminate data races and threading errors. This ensures that only error-free, optimized code is included in the performance analysis.

3.4. Data Race Detection with Intel Inspector

Intel Inspector [31] is utilized in this study as the primary tool for detecting memory and threading issues within the parallelized code generated by Large Language Models (LLMs). It is recognized for its high detection accuracy, especially in OpenMP applications, which is pertinent for evaluating the PolyBench benchmarks used in this study [32]. The study highlights the following key points about Intel Inspector:

- **High Detection Accuracy:** Intel Inspector excels at identifying true data races while minimizing false positives. This capability ensures that developers focus on genuine threading issues rather than being misled by erroneous reports.
- **Comprehensive Reporting:** The Intel Inspector tool provides detailed diagnostic information on various memory and threading issues, including data races, deadlocks, and memory leaks. This extensive reporting facilitates thorough error analysis and troubleshooting, making Intel Inspector a valuable asset for managing complex parallel applications.
- **Applicability to OpenMP:** With this study focusing on OpenMP parallelized codes, Intel Inspector’s specialization in detecting OpenMP-related issues is particularly advantageous. It outperforms several alternatives, such as ThreadSanitizer [14, 33], Helgrind [34] and Archer [35], in OpenMP-specific benchmarks by delivering more precise results.
- **Systematic Evaluation:** DataRaceBench benchmarks demonstrate that Intel Inspector offers consistent performance across a range of test cases, establishing it as a reliable tool for detecting issues in both standard and edge-case scenarios.

3.5. Functional Correctness

To evaluate the functional correctness of parallelized code, we first generate the output dump array from the sequential implementation of the benchmark in the PolyBench/C suite. This is achieved by compiling and executing the benchmark without any parallelization enabled. Next, we produce the output dump array for the parallelized version of the code, which is generated using compiler-assisted parallelization with OpenMP directives and LLM parallelized code. The outputs from both implementations are then compared using a line-by-line difference check with sequential code output. If the difference is zero, the parallelized code is deemed functionally correct, indicating that it preserves computational equivalence with the sequential baseline. Conversely, any nonzero difference suggests that the parallelized code is incorrectly implemented, highlighting potential issues introduced during parallelization.

3.6. Evaluation Metrics

We assess performance using three key metrics:

- **Speedup:** Calculated as the ratio of the serial execution time to the parallel execution time for each PolyBench benchmark. Speedup is computed for both ICC-parallelized code and LLM-generated code, allowing for direct comparison.
- **Scalability:** The benchmarks are run on various thread counts (2, 4, 8, 12, 16, 20, 24) to observe how performance scales with increasing parallelism.
- **Accuracy:** Accuracy is verified through ICC’s Inspector tool, ensuring that the LLM-generated parallel code is free from memory and threading errors. Codes that pass this validation are further evaluated against manually optimized ICC-parallelized versions to measure their computational efficiency.

By comparing the speedup and scalability of ICC and LLM-generated parallel code, we evaluate whether LLM-assisted code can achieve the same or better performance than traditional parallelization techniques.

4. EXPERIMENTAL SETUP

4.1. Toolchain and Environment

This section outlines the tools and software environment used during our experiments to parallelize PolyBench kernels and analyze their performance.

Compiler and Operating System. The experiments were conducted on a system running Ubuntu 20.04 LTS, utilizing the Intel C Compiler (icc) for both serial and parallel execution. The system hardware includes: (i) Processor: 1x ADL-S i9-12900 (16C, 8P+8E, 24T, 2.4/1.8G, 30M, 65W), (ii) Chipset: Intel W680, and (iii) RAM: 4x 32GB DDR5-4800 Non-ECC UDIMM (total 128GB). The system software used was Ubuntu 20.04 [36], icc 2021.7.1 20221019 [37] and Intel(R) Inspector 2024.1 (build 606844) [31] version for our experiments. We ran the benchmarks with different thread counts (2, 4, 8, 12, 16, 20, 24), comparing serial execution times to parallel executions.

Benchmarks. The PolyBench/C benchmark suite was employed to evaluate performance. The benchmark was compiled with the `-DPOLYBENCH_TIME` flag, which measures wall-clock time. The benchmark was executed multiple times to obtain a reliable measurement of its execution time, accounting for variability in system load and other factors. Typically, the script runs the benchmark five times, eliminates the two extremal times, and ensures that the deviation of the three remaining times does not exceed a 5% threshold.

This approach mitigates the impact of outliers caused by temporary system load spikes. Additionally, the script calculates the variance and standard deviation of the execution times to understand performance consistency. Variance measures the spread of execution times, while standard deviation provides the average deviation from the mean. The execution time may also be normalized to a reference time or baseline, facilitating comparison across different benchmarks or systems by dividing the measured time by the reference time to obtain a dimensionless ratio.

Selection of LLMs. We have a wide range of Large language models present nowadays and selecting any model for the experiment could be a difficult choice. We have selected 23 [38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60] LLMs for our experiments. For closed source models, we selected Claude-3.5-sonnet, Claude-3 (sonnet, haiku, opus), Gemini-pro (1, 1.5) and OpenAI-GPT models (3.5Turbo, 4o) based on model size, accessibility, and specific strengths in code generation and managing dependencies, as these models are recognized for their precision in structured code tasks and parallelization. For consistency and optimal accuracy, each model was configured with `top_p=1` and `temperature=0.0` to ensure deterministic outputs with minimal variability [61]. This selection process prioritized high-performing models within our computational limits, focusing on those most capable of generating reliable, parallelized code. For the open source models, we are bound by our system requirements, hence, we selected the models based on criteria as shown in Fig. 1.

Table 1 provides an overview of selected closed large language models (LLMs). It includes details such as the model name, knowledge cutoff date (KCD), model type (MT), a brief description, and key features. The table highlights the capabilities and specializations of each model, such as visual reasoning, high-speed processing, and advanced language comprehension, helping users understand the strengths and applications of different LLMs.

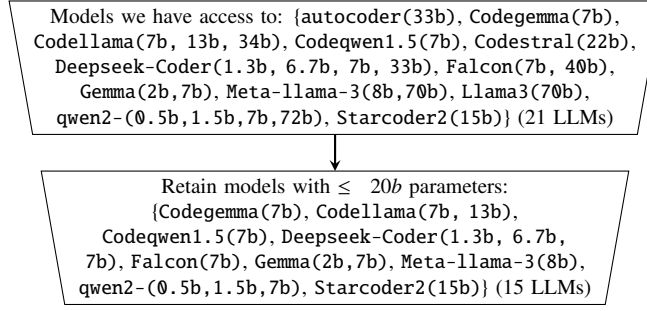


Figure 1: Our open source model selection process

Table 1: Selected Closed Large Language Model. MT stands for Model Type, TB stands for Transformer-Based, KCD stands for Knowledge Cutoff Date, and MLL stands for Multimodal LLM.

LLM Name	KCD	MT	Description	Key Features
claude-3-5-sonnet@20240620	2024-01	TB	Anthropic’s most advanced model, designed for enhanced vision capabilities and data analytics.	- Superior visual reasoning - Accurate text transcription from images - Ideal for data extraction from visuals
claude-3-haiku@20240307	2024-01	TB	Anthropic’s fastest model, offering a balance between speed and performance for various tasks.	- High-speed processing - Cost-effective - Suitable for real-time applications
claude-3-sonnet@20240229	2024-01	TB	Provides a combination of intelligence and speed, suitable for a wide range of applications.	- Balanced performance - Versatile applications - Efficient processing
claude-3-opus@20240229	2024-01	TB	Anthropic’s most capable model, offering advanced performance for complex tasks.	- Advanced capabilities - Handles complex tasks - High accuracy
gemini-1.0-pro-001	2023-04	MLL	Google’s large language model capable of handling multi-turn text and code interactions.	- Supports text and code chat - Code generation capabilities - Zero-shot, one-shot, few-shot learning
gemini-1.5-pro-001	2024-01	MLL	An enhanced version of Gemini 1.0 Pro with improved capabilities for text and code tasks.	- Advanced multimodal processing - Improved performance over 1.0 Pro - Suitable for complex applications
gpt-3.5-turbo-0125	2023-09	TB	OpenAI’s advanced language model optimized for conversational tasks and general-purpose use.	- High-quality text generation - Efficient performance - Suitable for chat applications
gpt-4o-2024-05-13	2024-04	TB	OpenAI’s latest language model with enhanced capabilities for complex reasoning and understanding.	- Improved reasoning abilities - Handles complex tasks - Advanced language comprehension

Table 2 provides an overview of selected open large language models (LLMs). It includes details such

as the model name, number of parameters (NoP), model type (MT), a brief description, and key features. The table highlights the capabilities and specializations of each model, such as code generation, debugging, and instruction-following, helping users understand the strengths and applications of different open-source LLMs in various programming and natural language processing (NLP) tasks.

Table 2: Selected Open Large Language Model. NoP stands for Number of Parameters, MT stands for Model Type, TD stands for Transformer Decoder, and CD stands for Causal Decoder

LLM Name	NoP	MT	Description	Key Features
CodeQwen1.5-7B-Chat	7B	TD	Specialized for code-focused chat interactions, assisting in code generation and debugging.	- Suitable for code chat apps - Effective for debugging - Enhances programming assistance
CodeLlama-13B-Instruct-hf	13B	TD	Tailored for advanced code generation and understanding, optimized for complex programming.	- Enhanced for complex tasks - Supports multiple languages - Suitable for complex code synthesis
CodeLlama-7B-Instruct-hf	7B	TD	Specialized in code synthesis and understanding with instruction-following capability.	- Supports multiple languages - Optimized for code tasks - Suitable for code generation
DeepSeek-Coder-1.3B-Instruct	1.3B	TD	Tailored for code generation tasks, fine-tuned for programming instructions.	- Suitable for code completion - Effective for programming instructions - Assists coding tasks
DeepSeek-Coder-6.7B-Instruct	6.7B	TD	Advanced code generation and understanding model for complex programming instructions.	- Suitable for advanced tasks - Effective for complex structures - Assists in programming tasks
DeepSeek-Coder-7B-Instruct-v1.5	7B	TD	Fine-tuned for code generation and comprehension in programming contexts.	- Effective for code tasks - Suitable for various programming need - Improved performance
Falcon-7B-Instruct	7B	CD	Fine-tuned on chat/instruct datasets, optimized for inference.	- Outperforms comparable open-source models of similar scale - Apache 2.0 license - Suitable for instruction tasks
CodeGemma-7B-It	7B	TD	Combines code generation and language understanding for programming tasks.	- Integrates code generation and - Suitable for programming tasks - Effective in code comprehension
Gemma-2B-It	2B	TD	Language model for various NLP tasks. Fine-tuned for text understanding.	- Focused - Effective in NLP - Suitable for Italian tasks
Gemma-7B-It	7B	TD	Model aimed at more complex NLP tasks with enhanced text processing.	- Advanced NLP model - Suitable for complex NLP - Effective in Italian text generation
Meta-Llama-3-8B-Instruct	8B	TD	Fine-tuned for instruction-based interactions across various NLP tasks.	- Versatile language model - Effective instruction following - Suitable for NLP applications
StarCoder2-15B-Instruct-v0.1	15B	CD	Optimized for coding-related instructions with single-turn instruction response.	- Optimized for code generation tasks - Efficient performance - Suitable for programming assistance
Qwen2-0.5B-Instruct	0.5B	TD	Small and efficient instruction-following model for language processing tasks.	- Lightweight and efficient - Suitable for smaller tasks - Effective in instruction following

Qwen2-1.5B-Instruct	1.5B	TD	Optimized for instruction tasks, balancing performance and computational efficiency.	- Suitable for a range of tasks - Effective instruction following - Balanced performance
Qwen2-7B-Instruct	7B	TD	Fine-tuned for instruction-based tasks, designed for handling complex language interactions.	- Suitable for complex NLP - Effective instruction following - Handles sophisticated tasks

Profiling Tools. Intel Inspector (inspxe-cl) [31] was used to check for memory and threading errors in the parallelized code. Only parallel code that passed these checks without errors was selected for further performance evaluation.

4.2. Compilation Flags

In this section, we outline the steps required to set up the environment, compile and parallelize code using ICC, generate parallel code with LLMs, and analyze the performance and errors of the generated code.

4.2.1. Compiling Serial Code

Benchmark kernels were compiled with icc’s optimization flags:

```
1 icc -O3 -I utilities -I <benchmark_directory> <source_code.c> -DPOLYBENCH_TIME -o <
  output_file>
```

4.2.2. Parallelizing Code with ICC

Benchmark kernels were compiled with icc’s auto-parallelization flags:

```
1 icc -parallel -O2 -par-num-threads=<num_threads> -I utilities -I <benchmark_directory>
  <source_code.c> -DPOLYBENCH_TIME -o <output_file>
```

4.2.3. Generating Parallel Code with LLMs

Use a pre-trained LLM by providing the sequential code as input with a prompt instructing it to generate equivalent OpenMP code. Prompt template:

```
1 You are a good sequential to parallel code generator. For the following C/C++ code,
  please generate a semantically equivalent OpenMP C/C++ code. Do not explain
  anything.
2 //begin of c code
3 {source_code}
4 //end of c code
```

The generated response by LLMs may have some extra text apart from the code, so we strip out that text. An example for that is as follows

```
1 ...
2 llm_response = ''
3 for response in responses:
4     llm_response += response.text
5 response = llm_response
6
7 if ("'"c" in response):
8     response = response.split("'"c")[1].split("'"')[0]
9 elif ("'"python" in response):
10    response = response.split("'"python")[1].split("'"')[0]
```

```

11 elif ("java" in response):
12     response = response.split("java")[1].split("```")[0]
13 elif response.count("```") == 2:
14     response = response.split("```")[1].split("```")[0]
15
16 if "```" in response:
17     response = response.strip("```")
18
19 # Ensure the code starts with a line containing #
20 lines = response.split('\n')
21 while lines and not lines[0].lstrip().startswith('#'):
22     lines.pop(0)
23 response = '\n'.join(lines)
24
25 return response.strip()
26 ...

```

4.2.4. Compiling LLM-Generated Code

Compile the generated OpenMP code using ICC:

```

1 icc -qopenmp -O3 -I utilities -I <benchmark_directory> <generated_code.c> -
  DPOLYBENCH_TIME -o <output_file>

```

4.2.5. Error Analysis

Run Intel Inspector to analyze the LLM-generated code for memory and threading issues:

```

1 inspxe-cl -collect mi3 -result-dir "./results_memory" ./<output_file>
2 inspxe-cl -collect ti3 -result-dir "./results_threading" ./<output_file>

```

4.2.6. Functional Correctness

Run the sequential, icc-parallelized code and LLM generated code to get the dump output array and find the diff.

```

1 icc -O3 -I utilities -I datamining/correlation utilities/polybench.c datamining/
  correlation/correlation.c -DPOLYBENCH_DUMP_ARRAYS -DEXTRALARGE_DATASET -o
  correlation_ref -lm
2 ./correlation_ref 2>correlation_ref_serial.out
3 icc -parallel -O3 -I utilities -I datamining/correlation utilities/polybench.c
  datamining/correlation/correlation.c -DPOLYBENCH_DUMP_ARRAYS -DEXTRALARGE_DATASET
  -o correlation_ref1 -lm
4 ./correlation_ref1 2>correlation_ref_icc.out
5 icc -qopenmp -O3 -I utilities -I datamining/correlation utilities/polybench.c
  datamining/correlation/correlation_LLM.c -DPOLYBENCH_DUMP_ARRAYS -
  DEXTRALARGE_DATASET -o correlation_ref2 -lm
6 ./correlation_ref2 2>correlation_ref_LLM.out
7 diff correlation_ref_icc.out correlation_ref.out | wc -l
8 diff correlation_ref_LLM.out correlation_ref.out | wc -l

```

5. EXPERIMENTAL RESULTS

In this section, we elaborate on answering our framed research questions for our evaluation.

5.1. Answering RQ1 (Parallelization and Concurrency Bugs): Common pitfalls, challenges, memory and thread error rates in LLM-generated parallel code

Motivation. The goal is to understand the limitations of large language models (LLMs) in generating correct parallel code, particularly in the context of parallel programming. By examining the types of errors these models frequently produce, we aim to identify common challenges that hinder the successful generation of parallel code. Additionally, we evaluate the reliability of LLM-generated code by analyzing the frequency of memory and threading errors, such as memory leaks, data races, or deadlocks.

Approach. We conducted a study where C code was generated using prompts provided to various LLMs. For this purpose, we tested both closed-source and open-source models. The experiment involved generating closed-source model code five times and open-source model 1 time for each request to determine if the model could produce the correct parallel code at least once. Closed-source models often prioritize aspects like safety, scalability, and performance, which can lead to inconsistent output. These inconsistencies arise from content filtering. As a result, even when using deterministic settings, subtle variations in the outputs may occur. Open-source models generally offer more consistent and predictable behavior. Since they lack the proprietary optimizations of closed-source models, their outputs tend to remain stable in controlled environments when the same deterministic settings are applied.

From the generated outputs, we assessed whether the LLMs failed to produce any code for the provided C code prompts. This helped us evaluate the success rates and identify common failure points. To further assess the quality of the LLM-generated code, we conducted two key analyses:

Memory Analysis (mi3): This detects common memory-related issues such as memory leaks, memory corruption, and illegal memory accesses.

Threading Analysis (ti3): This identifies threading-related issues, including data races, deadlocks, and thread contention.

Both of these analyses were conducted using Intel Inspector, a tool designed to pinpoint memory and threading problems in parallel code. By running these checks, we ensured that the LLM-generated code was free from critical concurrency bugs. Any code that exhibited at least one memory or threading issue was excluded from checking the correctness of output as it was deemed incorrectly parallelized and unreliable for performance evaluation. After using the diff method, as discussed earlier in the functional correctness, all codes had no memory or thread-related issues passed. Hence, only those parallelized codes are considered for final speedup calculation, which are free from both memory and thread error issues and have the same output as that of the serial code.

Results. Across 690 code generation requests from 23 different LLMs, a total of 667 successful code generations were achieved. However, in 23 cases, the LLMs failed to generate any code. The most common error encountered was the message: `Output blocked by content filtering policy`. Fig. 2 shows the data for the number of code generation and failure while generating code for each large language model. The grey bar shows successful code generation and the light grey bar shows failure cases where LLM is unable to generate any code.

Out of 667 parallelized code samples generated by LLMs, only 305 were successfully compiled. Among these, 90 code samples were entirely free of memory or threading errors after being analyzed with Intel Inspector. This indicates that a significant portion of LLM-generated code encounters concurrency issues, with a relatively small fraction being free from both memory and threading problems. Table 3 presents an analysis of memory and thread errors for various LLM (Large Language Model) names. It categorizes the errors into three types: both memory and thread errors (BMaTE), only memory errors (OME), and only thread errors (OTE). Additionally, it shows the counts of instances with no errors, compilation errors,

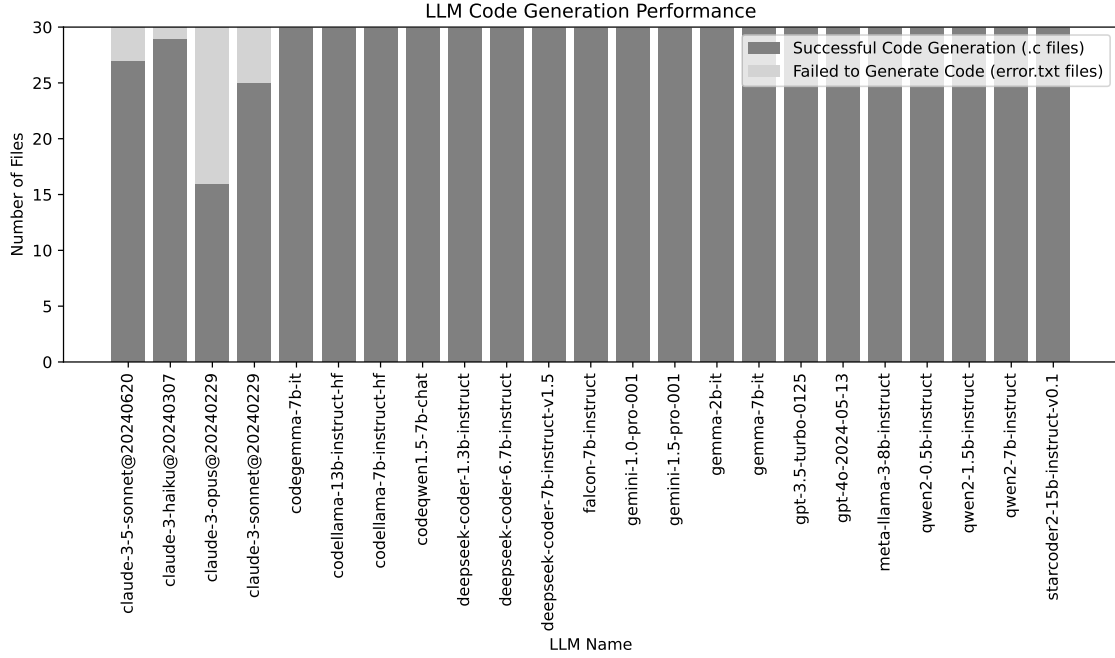


Figure 2: Performance of LLMs in generating code.

and the total represents the number of polybench kernels evaluated for each LLM. The integer value for the respective LLM in the table represents the number of kernels of polybench c codes that fall into that category. This analysis helps in understanding the error distribution and reliability of different LLMs in terms of memory and thread handling. Claude version does not have 30 codes in all cases because it was not able to generate C codes for all the cases as mentioned in Fig. 2, so we excluded them from execution.

Fig. 3 illustrates the distribution of different types of errors encountered during the performance evaluation of various LLMs in absolute numbers. It shows the counts of both memory and thread errors (BMaTE), only memory errors (OME), only thread errors (OTE), compilation errors, and instances with no errors. This chart provides a clear view of the frequency of each error type, highlighting that compilation errors are the most common, followed by only thread errors.

Fig. 4 presents the same error data but in percentages, offering a proportional view of the error distribution. It shows the relative occurrence of each error type as a percentage of the total errors. This chart emphasizes the predominance of compilation errors, which constitute more than half of the total errors, and provides a clearer understanding of the less frequent error types, such as only memory errors, which are highlighted with an arrow due to their small proportion

Table 3: Memory and thread error analysis. BMaTE stands for both memory and thread error, OME for only_memory_error and OTE for only_thread_error.

LLM_Name	BMaTE	OME	OTE	no_error	compilation_error	total
claude-3-5-sonnet@20240620	1	0	12	10	4	27
claude-3-haiku@20240307	2	0	15	3	9	29
claude-3-opus@20240229	0	0	11	4	1	16
claude-3-sonnet@20240229	0	0	4	2	19	25
codegemma-7b-it	2	0	1	0	27	30
codellama-13b-instruct-hf	3	1	9	13	4	30
codellama-7b-instruct-hf	0	0	1	2	27	30
codeqwen1.5-7b-chat	3	0	18	2	7	30
deepseek-coder-1.3b-instruct	0	0	1	1	28	30
deepseek-coder-6.7b-instruct	2	1	7	2	18	30
deepseek-coder-7b-instruct-v1.5	4	0	5	3	18	30
falcon-7b-instruct	0	0	0	1	29	30
gemini-1.0-pro-001	4	1	15	6	4	30
gemini-1.5-pro-001	0	1	13	14	2	30
gemma-2b-it	0	0	0	8	22	30
gemma-7b-it	1	0	17	1	11	30
gpt-3.5-turbo-0125	2	0	20	3	5	30
gpt-4o-2024-05-13	3	1	16	8	2	30
meta-llama-3-8b-instruct	0	0	2	1	27	30
qwen2-0.5b-instruct	0	0	0	2	28	30
qwen2-1.5b-instruct	2	0	3	2	23	30
qwen2-7b-instruct	2	1	6	2	19	30
starcode2-15b-instruct-v0.1	1	0	1	0	28	30

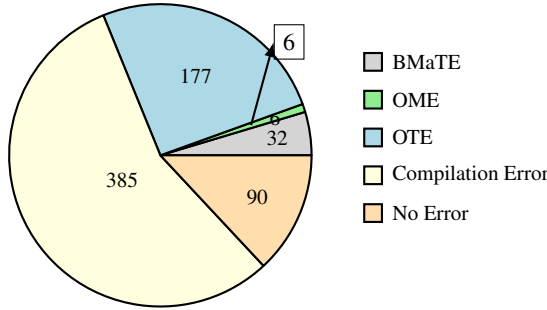


Figure 3: Pie chart of error types in absolute numbers.

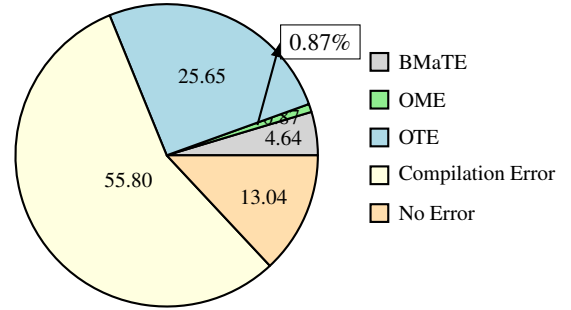


Figure 4: Pie chart of error types in percentages.

The primary challenge faced by LLMs in generating parallel code lies in content filtering, especially with closed-source models. While they excel in safety and scalability, these models may block valid outputs due to overly strict filtering policies. Open-source models, with their more stable and consistent behaviour, are less prone to such issues when used in controlled environments. Additionally, LLM-generated parallel code exhibits a notable rate of memory and threading errors, with only 90 out of 305 successfully compiled code samples passing all concurrency checks. While LLMs show potential in generating functional parallel code, the relatively high error rate highlights the need for further refinement in their ability to reliably handle complex memory and threading issues. State-of-the-art tools like Intel Inspector remain crucial in validating and debugging such code, ensuring correctness before performance evaluation.

5.2. Answering RQ2 (Speedup): Performance of LLM-generated parallel code

Motivation. The objective is to assess how the performance of parallel code generated by large language models (LLMs) compares with that of state-of-the-art parallelization tools, specifically ICC (Intel C Compiler). This comparison focuses on the speedup achieved by each method, with the goal of identifying LLMs that are capable of generating highly efficient parallel code.

Approach. We executed all generated code that successfully passed Inspector analysis, ensuring there were no threading or memory-related issues. For each benchmark, the code was run five times, and the two extremal (highest and lowest) times were discarded to reduce the influence of outliers caused by temporary system load spikes. The remaining three times were averaged, and we ensured that their deviation did not exceed 5%. This approach helped minimize the impact of system variability on performance measurements. Speedup was calculated by dividing the execution time of the serial code by the execution time of the LLM-parallelized code. If the resulting speedup was higher than that of the ICC-parallelized code, the LLM-generated code was considered more efficient. Otherwise, it was deemed less efficient.

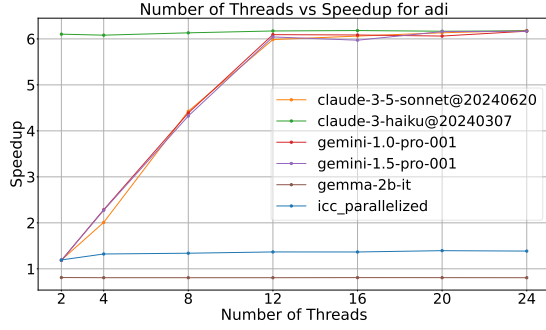
Results. The analysis of speedup values revealed mixed results for LLM-generated code. In some cases, LLMs were able to produce code that achieved notable speedup, though it was rare for the generated code to outperform ICC-parallelized code across the board. The kernel `3mm` achieved the highest speedup of $15.66\times$ using `icc_parallelized` code (`icc`) with 24 threads. LLM Performance: The kernel `gesummv` achieved a significant speedup of $7.5\times$ using `gemini-1.5-pro-001`(`gemini-1.5-pro`) with 16 threads, indicating that LLMs can be competitive in certain scenarios.

Fig. 5 shows notable LLM that give better speedup than state-of-art (`icc`) and are as follows:

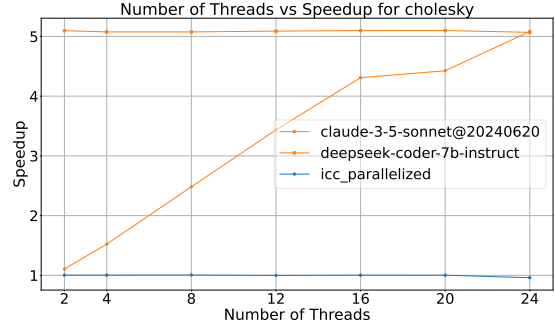
- `adi` achieved a significant speedup of $6.18\times$ using `claude-3-haiku@20240307`(`claude-3-haiku`) with 16 threads.
- `cholesky` achieved a speedup of $5.1\times$ using `deepseek-coder-7b-instruct`(`deepseek-cod-7b`) with 20 threads.
- `gemver` achieved a speedup of $5.82\times$ using `gemini-1.5-pro-001`(`gemini-1.5-pro`) with 24 threads.
- `jacobi-1d` achieved a speedup of $1.37\times$ using `qwen2-7b-instruct`(`qwen2-7b`) with 2 threads.
- `lu` achieved a speedup of $1.03\times$ using `qwen2-1.5b-instruct`(`qwen2-1.5b`) with 2 threads.
- `ludcmp` achieved a speedup of $1.06\times$ using `gemma-2b-it`(`gemma-2b`) with 4 threads.
- `mvt` achieved a speedup of $7.35\times$ using `gpt-4o-2024-05-13`(`gpt-4o`) with 24 threads.

Consistent ICC Performance: The ICC parallelization method consistently provided significant speedups across various kernels, such as:

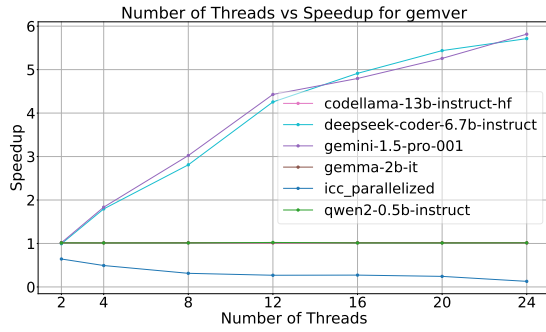
- `gemm` with a speedup of $7.65\times$ using 20 threads.
- `gramschmidt` with a speedup of $8.84\times$ using 8 threads.
- `symm` with a speedup of $11.95\times$ using 8 threads.
- `trmm` with a speedup of $9.13\times$ using 8 threads.



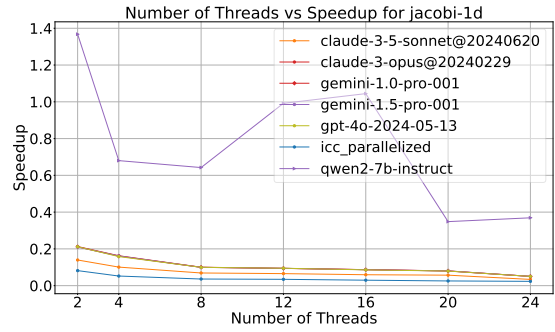
(a) Performance of adi



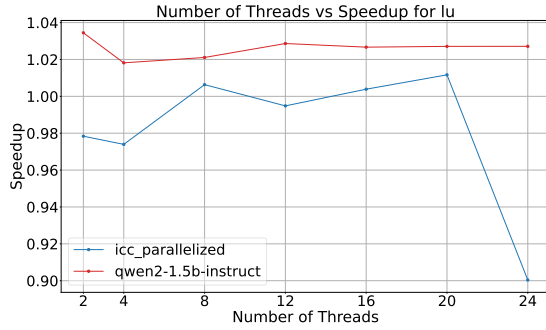
(b) Performance of cholesky



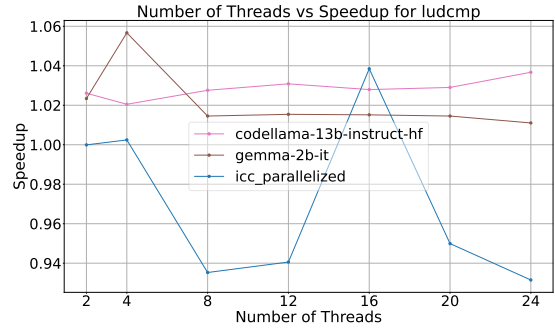
(c) Performance of gemver



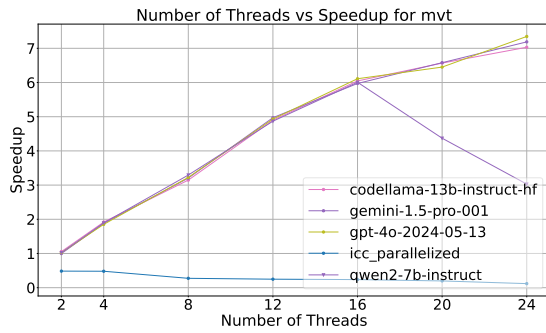
(d) Performance of jacobi-1d



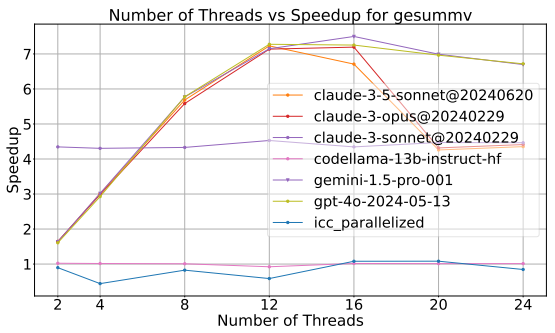
(e) Performance of lu



(f) Performance of ludcmp



(g) Performance of mvt



(h) Performance of gesummv

Figure 5: Kernels with better speedup due to LLM-based parallelization compared to the state-of-the-art (icc).

Lower Speedups: Some kernels, such as `atax` and `covariance`, achieved lower speedups of 0.13 $\times$ and 0.99 $\times$, respectively, indicating potential areas for optimization.

Table 4 presents the peak speedup obtained in our experiments for all 30 kernels in the PolyBench/C 4.2 benchmark suite. The table provides a comprehensive comparison of the maximum speedup achieved, the number of threads used, and the method of parallelization employed.

Table 4: Peak speedup obtained in our experiments for all 30 kernels in PolyBench/C 4.2

Kernel (benchmark)	Max. Speedup	#Threads	Parallelized by
2mm	1.62 $\times$	16	icc
3mm	15.66 $\times$	24	icc
adi	6.18 $\times$	16	claude-3-haiku
atax	0.13 $\times$	2	icc
bicg	1.51 $\times$	20	icc
cholesky	5.10 $\times$	20	deepseek-cod-7b
correlation	1.03 $\times$	16	icc
covariance	0.99 $\times$	4	icc
deriche	4.66 $\times$	20	icc
doitgen	1.00 $\times$	2	icc
durbin	1.06 $\times$	24	icc
fdtd-2d	5.10 $\times$	8	icc
floyd-warshall	1.04 $\times$	12	icc
gemm	7.65 $\times$	20	icc
gemver	5.82 $\times$	24	gemini-1.5-pro
gesummv	7.50 $\times$	16	gemini-1.5-pro
gramschmidt	8.84 $\times$	8	icc
heat-3d	5.38 $\times$	8	icc
jacobi-1d	1.37 $\times$	2	qwen2-7b
jacobi-2d	5.90 $\times$	8	icc
lu	1.03 $\times$	2	qwen2-1.5b
ludcmp	1.06 $\times$	4	gemma-2b
mvt	7.35 $\times$	24	gpt-4o
nussinov	1.10 $\times$	12	icc
seidel-2d	0.99 $\times$	12	icc
symm	11.95 $\times$	8	icc
syr2k	5.69 $\times$	12	icc
syrk	5.48 $\times$	16	icc
trmm	9.13 $\times$	8	icc
trisolv	0.96 $\times$	12	icc

Table 5 presents a comparison of various LLMs (Large Language Models) based on their performance in terms of speedup over serial code and state-of-the-art (SOTA) performance. It includes the count of cases where the speedup is greater than 1, the count of cases where the speedup is greater than the `icc` (Intel C Compiler) parallelized code performance, and the specific thread values that achieved these speedups. This helps in understanding which LLMs perform better in parallelized environments and the thread configurations that contribute to their performance. The numerical number in the second column shows the count of parallelized code by respective LLM where the speedup obtained is greater than 1, indicating that parallelized are more efficient than respective serial code. Similarly, the third column shows the count of parallelized code where the speedup achieved is greater than the respective `icc` parallelized code, and the last column shows the respective thread values where those speedups are obtained.

Table 5: Count of cases and respective thread value of LLMs performance comparison with SOTA (icc) and serial code time.

LLM_Name	#times speedup >1	#times speedup>ICC	#Threads
claude-3-5-sonnet@20240620	6	3	12,24,24
claude-3-haiku@20240307	1	1	16
claude-3-opus@20240229	1	1	16
claude-3-sonnet@20240229	1	1	12
codegemma-7b-it	0	0	0
codellama-13b-instruct-hf	7	2	24,4
codeqwen1.5-7b-chat	0	0	0
deepseek-coder-1.3b-instruct	0	0	0
deepseek-coder-6.7b-instruct	1	1	24
deepseek-coder-7b-instruct	1	1	20
falcon-7b-instruct	0	0	0
gemini-1.0-pro-001	4	1	24
gemini-1.5-pro-001	7	4	24,24,16,24
gemma-2b-it	4	2	8,4
gemma-7b-it	0	0	0
gpt-3.5-turbo-0125	0	0	0
gpt-4o-2024-05-13	3	2	24,12
meta-llama-3-8b-instruct	0	0	0
qwen2-0.5b-instruct	1	1	12
qwen2-1.5b-instruct	2	1	2
qwen2-7b-instruct	2	2	16,2
starcoder2-15b-instruct-v0.1	0	0	0

Table 6 provides a comparative analysis of various Large Language Models (LLMs) based on key performance metrics. It highlights the best thread configurations (BTC) for each model, their key strengths, and their overall rank. The `codellama-13b-instruct-hf` model ranks highest due to its balanced performance, fewer errors, and good speedup. The `gemini-1.5-pro-001` model follows closely, noted for its consistent speedup performance. Models like `gpt-4o-2024-05-13` and `claude-3-5-sonnet@20240620` are recognized for their high performance on specific kernels and consistent speedup, respectively. On the other hand, models such as `gpt-3.5-turbo-0125` and `codegemma-7b-it` are identified as underperformers due to poor speedup and significant compilation issues. The table serves as a useful guide for selecting LLMs under different conditions, emphasizing their strengths and optimal configurations.

Table 6: LLM Comparison Based on Metrics. BTC stands for best thread configuration.

LLM_Name	BTC	Key Strengths	Rank
codellama-13b-instruct-hf	24, 4	Balanced performance, fewer errors, good speedup	1
gemini-1.5-pro-001	24, 16	Best speedup consistency	2
gpt-4o-2024-05-13	24, 12	High performance on specific kernels	3
claude-3-5-sonnet@20240620	12, 24	Moderate errors, consistent speedup	4
gemma-2b-it	8, 4	Performs well under low threads	5
deepseek-coder-7b-instruct	20	High peak speedup in one kernel	6
claude-3-haiku@20240307	16	Moderate speedup	7
claude-3-opus@20240229	16	Stable but low performance	8
qwen2-7b-instruct	16, 2	Efficient on selective kernels	9
gemini-1.0-pro-001	24	Consistent but fewer scenarios	10
gpt-3.5-turbo-0125	N/A	Poor speedup and performance	11
codegemma-7b-it	0	Significant compilation issues	12
Others (low performance group)	N/A	Minimal contributions	N/A

Table 7 provides recommendations for selecting Large Language Models (LLMs) under various conditions based on their performance metrics. For scenarios requiring few errors and balanced speedup, `codellama-13b-instruct-hf` is recommended due to its consistent performance across most metrics.

`gemini-1.5-pro-001` is highlighted for achieving the best speedup over ICC, making it ideal for critical benchmarks. In environments with low thread configurations, `gemma-2b-it` is suggested for its efficiency in resource-constrained settings. For tasks requiring kernel-specific high speedup, `gpt-4o-2024-05-13` is recommended due to its strong performance on complex kernels. Lastly, for general exploration of parallelization, `claude-3-5-sonnet@20240620` is advised for its moderate errors and good speedup consistency.

Table 7: Recommendations for Selecting LLMs Under Different Conditions

Condition	Recommended LLM	Reason
Few Errors, Balanced Speedup Best Speedup Over ICC	<code>codellama-13b-instruct-hf</code> <code>gemini-1.5-pro-001</code>	Consistently good across most metrics. Highest performance on critical benchmarks.
Low Thread Configurations	<code>gemma-2b-it</code>	Efficient in resource-constrained environments.
Kernel-Specific High Speedup	<code>gpt-4o-2024-05-13</code>	Strong performance on specific complex kernels.
General Exploration of Parallelization	<code>claude-3-5-sonnet@20240620</code>	Moderate errors with good speedup consistency.

- *Best Overall Performer*:- `codellama-13b-instruct-hf` ranked the best due to fewer errors, consistent speedups (greater than 1), and occasional dominance over ICC performance.
- *High Speedup Consistency*:- `gemini-1.5-pro-001` demonstrated the highest count of significant speedups over ICC but had slightly more errors.
- *Specialized Performers*:- `gemma-2b-it` performed better with lower thread counts, making it a good choice for limited resource environments, `qwen2-7b-instruct` showed high efficiency in selective kernels but was inconsistent overall.
- *Under-performing Models*:- `gpt-3.5-turbo-0125` and `codegemma-7b-it` struggled significantly, showing high error counts and low speedups.
- *Errors vs. Speedup Trade-Off*:- Models like `gpt-4o-2024-05-13` showed potential on kernels requiring higher thread configurations but had higher error rates.

Across all tested LLMs, 26.66% of cases achieved a speedup surpassing that of the state-of-the-art Intel C Compiler (`icc`). Overall, the results demonstrate that while state-of-the-art (`icc`) remains a robust tool for parallelization, LLMs show promising potential in generating efficient parallel code for specific kernels. Further research and optimization could enhance the performance of LLM-generated code, making it a viable alternative to traditional compiler-based methods.

6. DISCUSSION

In this section, we discuss the actionable implications of our findings, and suggest potential downstream use cases for relevant stakeholders.

6.1. Actionable Insights

- While the internal logic of inferencing using open-source LLMs is not visible to the user, profiling LLM execution and binary analysis may provide promising insights as to why/how the serial code parallelization is performed.
- Instead of discarding parallelized source code due to zero-shot inferencing, iterative LLM response refinement with feedback from a conventional program analysis tool can be pursued.

6.2. Downstream Users

Software Developers. Developers looking to enhance the performance of computationally intensive applications through parallelization can leverage the findings and tools from this research.

High-Performance Computing (HPC) Researchers. Researchers in the field of HPC can use the methodologies developed in this study to optimize algorithms for better performance on multi-core architectures.

Compiler and Tools Developers. Engineers involved in compiler development can use insights from the performance comparisons between LLMs and traditional compilers to improve optimization techniques.

Academic Institutions. Universities and research institutions can adopt the study’s findings for educational purposes, teaching students about parallel programming and the capabilities of LLMs in code generation.

Industry Practitioners in Data Science and Machine Learning. Professionals in data science and machine learning can apply parallelization techniques to optimize data processing workflows, enhancing efficiency in model training and evaluation.

Embedded Systems Engineers. Engineers working on performance-sensitive applications in embedded systems can utilize parallel code generation techniques to maximize efficiency and resource utilization.

Cloud Computing Providers. Providers of cloud computing services can integrate LLM-generated parallelization techniques into their offerings, enabling users to automatically optimize applications for better performance.

7. THREATS TO VALIDITY

External Validity. The findings of our study may not be applicable to all programming languages, paradigms, or types of applications. The study primarily utilizes the PolyBench benchmark suite, which might not represent a diverse range of real-world scenarios, limiting the generalization of results. Additionally, the specific hardware configuration (Intel i9-12900, Ubuntu 20.04) used for experiments could lead to variability in outcomes when applied to different systems or environments. Rapid advancements in large language models (LLMs) and compiler technologies could also render the results obsolete, as new models may exhibit significantly different performance characteristics.

Construct Validity. The metrics employed for evaluation, such as speedup and error rates, may not fully encapsulate the performance and correctness of the generated code. Important aspects like code maintainability and readability were not considered, potentially misrepresenting the overall effectiveness of LLMs. Furthermore, the selection of 23 pre-trained LLMs may not encompass the full spectrum of available models, introducing sampling bias that could skew conclusions about LLM capabilities in automatic code parallelization. Our work relies solely on simple prompting and does not involve access to the internal details of the language models, hence, conducting an ablation study to attribute specific properties of the models to the observed improvements is not feasible. However, despite this limitation, we believe the results presented here are meaningful and stand on their own merit.

Internal Validity. Internal validity is threatened by potential confounding variables that may influence the outcomes of the study. Differences in optimization strategies between the Intel C Compiler (ICC) and LLMs could introduce biases in the conclusions drawn regarding their respective efficiencies. The reliance on average execution times for performance measurement could overlook variability due to system load or resource contention, further complicating the interpretation of results. The cumulative effects of errors detected by Intel Inspector may not be adequately captured, potentially underestimating the true error rates of the generated code. As with any automated tool, there is a possibility of false positives or missed detections. To enhance the accuracy of the evaluation, in the future we plan to incorporate a human review of a sampled set of generated code to validate the tool’s findings. The randomness of LLM responses at model temperature zero, subjected to a repetition factor of five which may not be sufficient. However, this is largely the case with two closed source models (ChatGPT from OpenAI, Claude from Anthropic) with costly API calls. In the future, we may need to increase this factor to 30 as suggested in [62] for randomized experiments.

8. FUTURE WORK

Future research can significantly advance the field of automatic code parallelization, making LLMs a more effective and reliable tool for software developers. Several directions for future work are mentioned as follows.

Prompt Feedback. In this work, we focused on detecting memory and threading errors in automatically parallelized code using tools like Intel Inspector. An interesting and promising extension of this approach is the integration of a feedback loop [9] where identified errors are fed back into the language model to refine its initial suggestions. While language models already demonstrate strong performance, incorporating such iterative feedback mechanisms could further enhance their accuracy and robustness. We recognize the potential of this approach and highlight it as an avenue for future research to improve the reliability of automatically parallelized programs.

Expanding Benchmark Diversity. Future research can explore a broader range of benchmarks beyond the PolyBench suite to evaluate the performance of LLMs in various real-world applications. Incorporating different domains, such as graphics processing, data analytics, and machine learning, could provide a more comprehensive assessment of LLM capabilities in code parallelization.

Refining LLM Architectures. Continued development of LLMs specifically tailored for code generation and parallelization could enhance their performance. Research can focus on fine-tuning existing models or creating new architectures that better understand parallel programming concepts, such as data dependencies and synchronization mechanisms.

Integrating Verification Tools. Enhancing the integration of formal verification and static analysis tools alongside LLMs could improve the reliability of generated code. Future work could involve developing frameworks that automatically validate and correct LLM-generated code for memory and threading issues, ensuring higher accuracy and safety in parallel applications.

User-Friendly Interfaces for Developers. Developing user-friendly tools and interfaces that allow developers to interact with LLMs for code parallelization could facilitate broader adoption. Future work could focus on creating intuitive platforms that guide users through the process of transforming serial code into parallel code, making it accessible to a wider audience.

Performance Benchmarking Across Architectures. Future research can investigate the performance of LLM-generated parallel code across various hardware architectures, such as GPUs and TPUs. Understanding how different architectures impact performance will be crucial for optimizing code generation and ensuring scalability.

9. CONCLUSION

This research has explored the potential of large language models (LLMs) for automatic code parallelization, specifically within the context of the PolyBench benchmark suite. By comparing the performance of 23 pre-trained LLMs against the state-of-the-art Intel C Compiler (ICC), we have demonstrated that LLM-generated parallel code can achieve competitive speedups, with some instances outperforming ICC in non-functional aspects such as execution speed. Our findings reveal that while LLMs show promise as tools for generating performant parallel code, they also encounter significant challenges, particularly in correctly identifying and managing parallel programming nuances, such as data dependencies and synchronization mechanisms. The analysis of memory and threading issues using Intel Inspector has highlighted areas where LLMs struggle, emphasizing the need for robust validation and error detection to ensure the reliability of automatically generated code. The study contributes valuable insights into the comparative effectiveness of LLMs and traditional compilers in code parallelization. It opens up new avenues for future research, including the development of more specialized LLM architectures, the integration of advanced verification tools, and the exploration of hybrid approaches that combine compiler optimizations with AI-driven code generation. Ultimately, this research lays the groundwork for further exploration into the capabilities of LLMs in software development, particularly in high-performance computing environments. As the landscape of AI and compiler technology continues to evolve, leveraging LLMs for automatic code parallelization may become an essential tool in the quest for efficient and reliable software solutions.

Data Statement. Our artifacts include automated scripts, data, and code generated by LLMs from OpenAI. These are currently under review by the concerned OpenAI team, and will be made public upon approval.

Author contributions: CRediT. *First author:* Conceptualization; Data curation; Methodology; Software; Validation; Visualization; Formal analysis; Writing – original draft. *Second author:* Funding acquisition; Investigation; Project administration; Methodology; Resources; Supervision; Formal analysis; Writing – review and editing.

Funding Sources. This work was supported by IIT Gandhinagar (grant no. IP/IITGN/CSE/SM/2324/02), Google (grant no. GCP297941264), and OpenAI API Researcher Access Program (grant no. 0000004087).

References

- [1] L.-N. Pouchet, et al., Polybench: The polyhedral benchmark suite, *International Journal of High Performance Computing Applications* (2012) 1–15.
- [2] Intel Corporation, Intel C++ Compiler Documentation (2020).
- [3] J. Jiang, F. Wang, J. Shen, S. Kim, S. Kim, [A survey on large language models for code generation](#) (2024). [arXiv:2406.00515](#).
URL <https://arxiv.org/abs/2406.00515>
- [4] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, H. Wang, [Large language models for software engineering: A systematic literature review](#) (2024). [arXiv:2308.10620](#).
URL <https://arxiv.org/abs/2308.10620>
- [5] D. Nichols, J. H. Davis, Z. Xie, A. Rajaram, A. Bhatele, [Can large language models write parallel code?](#), in: *Proceedings of the 33rd International Symposium on High-Performance Parallel and Distributed Computing, HPDC '24*, Association for Computing Machinery, New York, NY, USA, 2024, p. 281–294. [doi:10.1145/3625549.3658689](#).
URL <https://doi.org/10.1145/3625549.3658689>
- [6] J. Shi, Z. Yang, D. Lo, [Efficient and green large language models for software engineering: Literature review, vision, and the road ahead](#) (2024). [arXiv:2404.04566](#).
URL <https://arxiv.org/abs/2404.04566>
- [7] M. Mišić, M. Dodović, [An assessment of large language models for openmp-based code parallelization: a user perspective](#), *Journal of Big Data* 11 (1) (2024) 161.
URL <https://doi.org/10.1186/s40537-024-01019-z>
- [8] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, D. Zhou, Chain-of-thought prompting elicits reasoning in large language models, in: *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*, Curran Associates Inc., Red Hook, NY, USA, 2024.
- [9] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, A. Chadha, [A systematic survey of prompt engineering in large language models: Techniques and applications](#) (2024). [arXiv:2402.07927](#).
URL <https://arxiv.org/abs/2402.07927>
- [10] M. J. Quinn, *Parallel Programming in C with MPI and OpenMP*, McGraw-Hill, New York, 2004.
- [11] T. Rauber, G. Rünger, *Parallel Programming: For Multicore and Cluster Systems*, 2nd Edition, Springer, New York, 2010.
- [12] M. E. Wolf, *High-Performance Compilers for Parallel Computing*, Addison-Wesley, Reading, MA, 1996.
- [13] S. S. Muchnick, *Advanced Compiler Design and Implementation*, Morgan Kaufmann, 1997.
- [14] K. Serebryany, D. Bruening, A. Potapenko, D. Vyukov, Threadsanitizer: A fast data race detector, in: *Proceedings of the 2012 USENIX Annual Technical Conference (USENIX ATC)*, 2012, pp. 305–316.

- [15] L.-N. Pouchet, [Polybench/c: The polyhedral benchmark suite](https://github.com/MatthiasJReisinger/PolyBenchC-4.2.1), version 4.2.1 (2012).
URL <https://github.com/MatthiasJReisinger/PolyBenchC-4.2.1>
- [16] D. Yadav, S. Mondal, [Artifacts] This will be made public upon approval from OpenAI (Dec 2024).
- [17] P. Colella, Defining software requirements for scientific and data-intensive applications, *International Journal of High Performance Computing Applications* 18 (3) (2004) 392–399.
- [18] L.-N. Pouchet, et al., Polyhedral model and its applications to program optimization, in: *Proceedings of the International Conference on Compiler Construction*, 2011, pp. 209–228.
- [19] P. Feautrier, C. Lengauer, Polyhedral parallelism revisited, *The Journal of Supercomputing* 56 (3) (2011) 319–342.
- [20] A. Nicolau, et al., Automatic parallelization of fortran programs in the presence of complex control flow, in: *Proceedings of the 1991 ACM/IEEE Conference on Supercomputing*, 1991, pp. 682–691.
- [21] L. Dagum, R. Menon, Openmp: an industry standard api for shared-memory programming, *IEEE Computational Science and Engineering* 5 (1) (1998) 46–55.
- [22] C. Bastoul, et al., Evaluating automatic parallelization of polybench kernels, in: *Proceedings of the International Workshop on Polyhedral Compilation Techniques*, 2016.
- [23] S. Fang, et al., Scalability of automatic parallelization on many-core systems, *International Journal of Parallel Programming* 37 (2009) 475–491.
- [24] W. Joubert, et al., Automatic parallelization for multi-core systems: A case study, *Concurrency and Computation: Practice and Experience* 22 (12) (2010) 1494–1512.
- [25] C. Lattner, V. Adve, Llvm: A compilation framework for lifelong program analysis and transformation, *International Symposium on Code Generation and Optimization* (2004) 75–86.
- [26] A. J. Bik, D. Gannon, Automated data race detection and prevention in parallel programs, in: *Proceedings of the 1997 ACM/IEEE Conference on Supercomputing*, 1997, pp. 1–10.
- [27] K. Chen, et al., Exploring large language models for code generation and parallelization, in: *Proceedings of the 2023 ACM Symposium on Applied Computing*, 2023.
- [28] H. Touvron, et al., Data handling challenges in llm code generation, *ACM Transactions on Software Engineering and Methodology* (2023).
- [29] B. Freitag, et al., Towards robust verification of code generated by llms, *IEEE Software* (2022).
- [30] J. L. Hennessy, D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 5th Edition, Morgan Kaufmann, 2011.
- [31] Intel Corporation, Intel Inspector, Version 2024.1 (Build 606844), available at <https://www.intel.com/content/www/us/en/developer/tools/oneapi/inspector.html> (2024).
- [32] C. Liao, P.-H. Lin, J. Asplund, M. Schordan, I. Karlin, Dataracebench: a benchmark suite for systematic evaluation of data race detection tools, in: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2017, pp. 1–14.

- [33] Google, [ThreadSanitizer: A Data Race Detector](#), available as part of LLVM/Clang and GCC (2021).
URL <https://clang.llvm.org/docs/ThreadSanitizer.html>
- [34] N. Nethercote, J. Seward, [Helgrind: A Data Race Detector](#), part of the Valgrind Tool Suite (2007).
URL <https://valgrind.org/docs/manual/hg-manual.html>
- [35] J. Protze, A. Supalov, C. Terboven, M. S. Mueller, R. Dietrich, Archer: Effectively spotting data races in large openmp applications, in: Proceedings of the International Workshop on OpenMP (IWOMP), Vol. 8766 of Lecture Notes in Computer Science, Springer, 2014, pp. 53–67. doi: [10.1007/978-3-319-11454-5_5](#).
- [36] Canonical Ltd., Ubuntu 20.04 LTS, available at <https://releases.ubuntu.com/20.04/> (2020).
- [37] Intel Corporation, Intel C++ Compiler, Version 2021.7.1 (Build 20221019), available as part of Intel oneAPI, <https://www.intel.com/content/www/us/en/developer/tools/oneapi/dpc-compiler.html> (2022).
- [38] Anthropic, Claude-3.5-sonnet Model Documentation, available at <https://www.anthropic.com/claude> (2023).
- [39] Anthropic, Claude-3-sonnet Model Documentation, available at <https://www.anthropic.com/claude> (2023).
- [40] Anthropic, Claude-3-haiku Model Documentation, available at <https://www.anthropic.com/claude> (2023).
- [41] Anthropic, Claude-3-opus Model Documentation, available at <https://www.anthropic.com/claude> (2023).
- [42] C. Team, CodeGemma 7B Model Documentation, available at <https://codegemma.ai/docs> (2023).
- [43] B. Roziere, et al., [Code llama: Open foundation models for code - 7b version](#), arXiv preprint arXiv:2308.12950 (2023).
URL <https://arxiv.org/abs/2308.12950>
- [44] B. Roziere, et al., [Code llama: Open foundation models for code - 13b version](#), arXiv preprint arXiv:2308.12950 (2023).
URL <https://arxiv.org/abs/2308.12950>
- [45] Alibaba DAMO Academy, Qwen-1.5 7B: Foundation Models by Alibaba DAMO Academy, available at <https://modelscope.cn/models> (2023).
- [46] D. AI, DeepSeek-Coder 1.3B Model Documentation, available at <https://deepseek.com/models/coder> (2023).
- [47] D. AI, DeepSeek-Coder 6.7B Model Documentation, available at <https://deepseek.com/models/coder> (2023).
- [48] D. AI, DeepSeek-Coder 7B Model Documentation, available at <https://deepseek.com/models/coder> (2023).
- [49] Technology Innovation Institute, [Falcon llm 7b: Foundation models for open-source applications](#), arXiv preprint arXiv:2306.01116 (2023).
URL <https://arxiv.org/abs/2306.01116>

- [50] G. A. Team, Gemma 2B Model Documentation, available at <https://gemma.ai/models> (2023).
- [51] G. A. Team, Gemma 7B Model Documentation, available at <https://gemma.ai/models> (2023).
- [52] DeepMind, Gemini-pro 1 Model Documentation, available at <https://www.deepmind.com/publications> (2023).
- [53] DeepMind, Gemini-pro 1.5 Model Documentation, available at <https://www.deepmind.com/publications> (2023).
- [54] OpenAI, GPT-3.5 Turbo Model Documentation, available at <https://platform.openai.com/docs/models> (2023).
- [55] OpenAI, GPT-4o Model Documentation, available at <https://platform.openai.com/docs/models> (2023).
- [56] H. Touvron, M. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Roziere, N. Goyal, E. Hambro, F. Azhar, et al., [Llama: Open and efficient foundation language models - 8b version](#), arXiv preprint arXiv:2302.13971 (2023).
URL <https://arxiv.org/abs/2302.13971>
- [57] Alibaba DAMO Academy, Qwen2 0.5B Model Documentation, available at <https://modelscope.cn/models> (2023).
- [58] Alibaba DAMO Academy, Qwen2 1.5B Model Documentation, available at <https://modelscope.cn/models> (2023).
- [59] Alibaba DAMO Academy, Qwen2 7B Model Documentation, available at <https://modelscope.cn/models> (2023).
- [60] R. Li, A. Belouadi, A. Casanova, J. Chaumond, G. Lample, T. Scialom, [Starcoder2: Foundation models for code - 15b version](#), arXiv preprint arXiv:2305.06161 (2023).
URL <https://arxiv.org/abs/2305.06161>
- [61] A. Holtzman, J. Buys, L. Du, M. Forbes, Y. Choi, [The curious case of neural text degeneration](#), arXiv preprint arXiv:1904.09751 (2019).
URL <https://arxiv.org/abs/1904.09751>
- [62] A. Arcuri, L. Briand, [A practical guide for using statistical tests to assess randomized algorithms in software engineering](#), in: Proceedings of the 33rd International Conference on Software Engineering, ICSE '11, Association for Computing Machinery, New York, NY, USA, 2011, p. 1–10. doi:[10.1145/1985793.1985795](https://doi.org/10.1145/1985793.1985795).
URL <https://doi.org/10.1145/1985793.1985795>